

Hashing

Dr. Md Shamsuzzoha Bayzid

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology



Hashing: Motivation

What is the time complexity for searching an element in an array or linked list?

Hashing: Motivation

What is the time complexity for searching an element in an array or linked list?

$O(n)$

$O(\log n)$ for binary search

Hashing: Motivation

Many applications require faster lookup/search

1. Searching for an NID in the national NID database
2. Searching for a phone number in the phone database

Hashing

$O(1)$ store and lookup

Idea behind Hash Tables

Hash function

Store data (key) using a hash value. This hash values is computed based on the data (key).

Idea behind Hash Tables

Imagine you have to store some integers using the following hash function.

$$h(k) = k$$

	1		3			6		8
--	---	--	---	--	--	---	--	---

Store 1, 3, 6, and 8

Idea behind Hash Tables

It works great: $O(1)$
But do you see any problem?

Idea behind Hash Tables

It works great: $O(1)$

But do you see any **problem**?

You need to use a large table.
Many indices in the array will
remain empty

Hash function

Hash function

Map large key space to small table index

For a table of size 10:

$$h(k) = k \bmod 10$$

Hash function

Hash function

Map large key space to small table index

For a table of size 10:

$$h(k) = k \bmod 10$$

Collision

Multiple keys may produce the same hash value

11, 21, 31, etc. will map to index 1

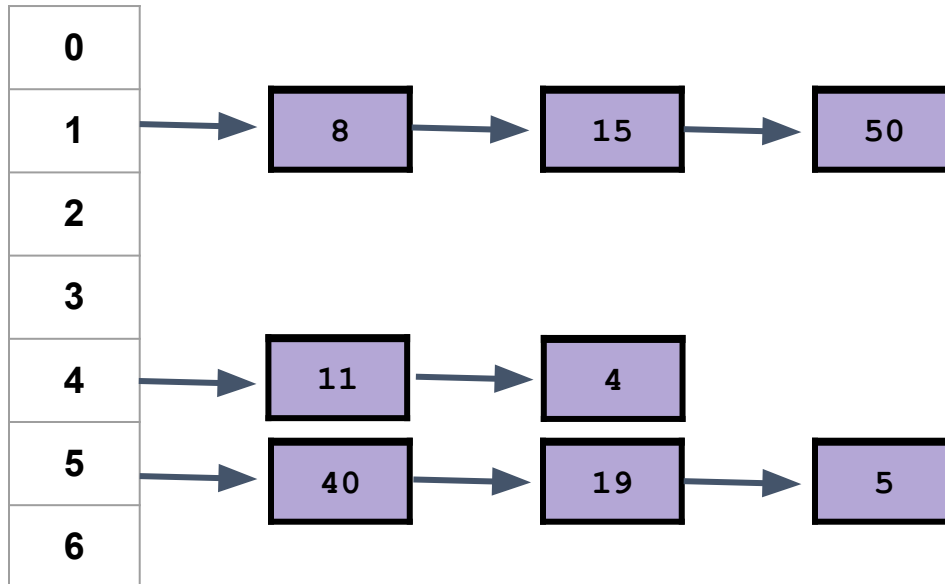
Hashing techniques: Collision resolution

Chaining

Each slot in the table stores a linked list

Store 8, 11, 4, 15, 40, 19, 50, 5

$$h(k) = k \bmod 7$$



Hashing techniques: Open addressing

Open Addressing

All elements are stored within the table. **If collision occurs,**
probe for an **empty slot** in the hash table

Open Addressing: linear probing

Stores data in the next available slot

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

[illegible]

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
								8	

$$h(8) = 8 \bmod 10 = 8$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
	11							8	

$$\begin{aligned}h(8) &= 8 \bmod 10 = 8 \\h(11) &= 11 \bmod 10 = 1\end{aligned}$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
	11			4				8	

$$\begin{aligned}h(8) &= 8 \bmod 10 = 8 \\h(11) &= 11 \bmod 10 = 1 \\h(4) &= 4 \bmod 10 = 4\end{aligned}$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
	11			4	15			8	

$$\begin{aligned}h(8) &= 8 \bmod 10 = 8 \\h(11) &= 11 \bmod 10 = 1 \\h(4) &= 4 \bmod 10 = 4 \\h(15) &= 15 \bmod 10 = 5\end{aligned}$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
40	11			4	15			8	

$$\begin{aligned}h(8) &= 8 \bmod 10 = 8 \\h(11) &= 11 \bmod 10 = 1 \\h(4) &= 4 \bmod 10 = 4 \\h(15) &= 15 \bmod 10 = 5 \\h(40) \bmod 10 &= 0\end{aligned}$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
40	11			4	15			8	19

$$\begin{aligned}h(8) &= 8 \bmod 10 = 8 \\h(11) &= 11 \bmod 10 = 1 \\h(4) &= 4 \bmod 10 = 4 \\h(15) &= 15 \bmod 10 = 5 \\h(40) \bmod 10 &= 0 \\h(19) \bmod 10 &= 9\end{aligned}$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5

$$h(k) = k \bmod 10$$

0	1	2	3	4	5	6	7	8	9
40	11			4	15			8	19

$$h(8) = 8 \bmod 10 = 8$$

$$h(11) = 11 \bmod 10 = 1$$

$$h(4) = 4 \bmod 10 = 4$$

$$h(15) = 15 \bmod 10 = 5$$

$$h(40) \bmod 10 = 0$$

$$h(19) \bmod 10 = 9$$

$$h(50) = 50 \bmod 10 = 0$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5

$$h(k) = k \bmod 10$$

0	1	2	3	4	5	6	7	8	9
40	11			4	15			8	19



Collision

$$\begin{aligned}h(8) &= 8 \bmod 10 = 8 \\h(11) &= 11 \bmod 10 = 1 \\h(4) &= 4 \bmod 10 = 4 \\h(15) &= 15 \bmod 10 = 5 \\h(40) \bmod 10 &= 0 \\h(19) \bmod 10 &= 9 \\h(50) &= 50 \bmod 10 = 0\end{aligned}$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
40	11	50		4	15			8	19



Collision



**Next available
slot**

$$h(8) = 8 \bmod 10 = 8$$

$$h(11) = 11 \bmod 10 = 1$$

$$h(4) = 4 \bmod 10 = 4$$

$$h(15) = 15 \bmod 10 = 5$$

$$h(40) \bmod 10 = 0$$

$$h(19) \bmod 10 = 9$$

$$h(50) = 50 \bmod 10 = 0$$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5
 $h(k) = k \bmod 10$

0	1	2	3	4	5	6	7	8	9
40	11	50		4	15			8	19

$h(8) = 8 \bmod 10 = 8$
 $h(11) = 11 \bmod 10 = 1$
 $h(4) = 4 \bmod 10 = 4$
 $h(15) = 15 \bmod 10 = 5$
 $h(40) \bmod 10 = 0$
 $h(19) \bmod 10 = 9$
 $h(50) = 50 \bmod 10 = 0$
 $h(5) = 5 \bmod 10 = 5$

Hashing techniques: Open addressing

Open Addressing: linear probing

Stores data in the next available slot

Store 8, 11, 4, 15, 40, 19, 50, 5

$$h(k) = k \bmod 10$$

0	1	2	3	4	5	6	7	8	9
40	11	50		4	15	5		8	19

$$h(8) = 8 \bmod 10 = 8$$

$$h(11) = 11 \bmod 10 = 1$$

$$h(4) = 4 \bmod 10 = 4$$

$$h(15) = 15 \bmod 10 = 5$$

$$h(40) \bmod 10 = 0$$

$$h(19) \bmod 10 = 9$$

$$h(50) = 50 \bmod 10 = 0$$

$$h(5) = 5 \bmod 10 = 5$$

Hashing techniques: Open addressing

Open Addressing: double hashing

Use a **second hash function** to **jump** from the original slot where the collision occurred

Linear probing can be viewed as a double hashing with $h_2(k) = 1$

Do you see why?